Ponto De Conhecimento Tecnológico

HANDS-ON #06

SQL Server — Joins, CTEs e Window Functions

Documentação técnica | Série 2 — SQL Server

Documento estruturado nas seis secções aprovadas. Base demonstrativa: SIManagerDB — Gestão de Serviços e Aplicações.

1. Introdução e Objetivos

1.1. Enquadramento

O HANDS-ON #06 — Joins, CTEs e Window Functions integra a Série 2 — SQL Server, promovida pelo Ponto De Conhecimento Tecnológico, uma iniciativa de partilha de conhecimento técnico através de exercícios práticos, provas de conceito e documentação de soluções tecnológicas.

Este Hands-On dá continuidade ao HANDS-ON #05 — Modelação de Dados e Constraints, aprofundando a exploração e análise de dados relacionais através da linguagem T-SQL (Transact-SQL). Enquanto o Hands-On anterior incidiu sobre a estrutura relacional, os tipos de dados, as constraints e a integridade da informação, o presente exercício concentra-se na construção de consultas SQL para responder a necessidades funcionais e analíticas.

O desenvolvimento utiliza a base de dados demonstrativa SIManagerDB — Gestão de Serviços e Aplicações, constituída por 15 tabelas funcionais e dados inteiramente fictícios, representativos de um ambiente organizacional de gestão de sistemas de informação. Os exercícios foram implementados e executados no Microsoft SQL Server, recorrendo ao SQL Server Management Studio (SSMS), e acompanhados de validações técnicas destinadas a confirmar a coerência dos resultados.

1.2. Objetivo Geral

Demonstrar a utilização integrada de Joins, Common Table Expressions (CTEs), agregações e funções de janela (Window Functions) na construção de consultas T-SQL orientadas para cenários reais de gestão e análise de informação. Pretende-se evidenciar não apenas a sintaxe SQL, mas também a relação entre os requisitos funcionais, o modelo de dados, a lógica de consulta e a validação dos resultados obtidos.

1.3. Objetivos Específicos

• Explorar relacionamentos entre tabelas com INNER JOIN e LEFT JOIN, preservando registos sem correspondência opcional.

• Estruturar consultas com CTEs, separando preparação, agregação e apresentação dos resultados.

• Produzir indicadores agregados com GROUP BY e COUNT().

• Identificar os registos mais recentes por entidade com ROW_NUMBER() e PARTITION BY, usando ordenação determinística.

• Calcular indicadores sem eliminar o detalhe dos registos, com COUNT() OVER().

• Construir classificações e analisar empates com RANK() e DENSE_RANK().

• Analisar séries temporais com SUM() OVER(), LAG() e LEAD().

• Garantir continuidade das séries mensais através de um calendário mensal.

• Validar tecnicamente os resultados e reconciliar os totais com a SIManagerDB.

• Promover legibilidade, organização, integridade e separação da lógica funcional e da validação.

1.4. Âmbito Funcional

Exercício	Cenário funcional	Técnicas principais
01	Aplicações e departamentos responsáveis	INNER JOIN
02	Incidentes com e sem técnico atribuído	LEFT JOIN
03	Indicadores por aplicação	CTE, GROUP BY, COUNT()
04	Últimos três incidentes por aplicação	ROW_NUMBER(), PARTITION BY, COUNT() OVER()
05	Ranking por volume de incidentes	RANK(), DENSE_RANK()
06	Evolução mensal dos incidentes	CTE, SUM() OVER(), LAG(), LEAD()

Os exercícios foram concebidos de forma progressiva, começando pelas operações de junção entre tabelas e evoluindo para consultas analíticas com funções de janela. A abordagem privilegia a aplicação das técnicas SQL a problemas funcionais concretos.

1.5. Ambiente Tecnológico

Componente	Tecnologia
Sistema de gestão de bases de dados	Microsoft SQL Server 2019 ou superior
Ferramenta de desenvolvimento	SQL Server Management Studio (SSMS)
Linguagem	T-SQL
Base de dados	SIManagerDB
Domínio funcional	Gestão de Serviços e Aplicações
Dados utilizados	Inteiramente fictícios
Tipo de operações	Consultas de leitura e análise
Organização	Seis scripts SQL independentes

Os scripts utilizam instruções de consulta e não efetuam alterações aos dados existentes na SIManagerDB. A demonstração complementar do Exercício 05 utiliza valores definidos diretamente em T-SQL, sem modificar as tabelas.

1.6. Metodologia de Desenvolvimento e Validação

• Desenho técnico: identificação de objetivos funcionais, tabelas, relacionamentos e técnicas SQL.

• Implementação: desenvolvimento progressivo e execução dos seis exercícios no SSMS.

• Validação técnica: consultas complementares para confirmar contagens, correspondências, agregações, posições, classificações e acumulados.

• Consolidação: organização de consultas principais e validações em seis ficheiros .sql, comentados em português de Portugal, e reexecução dos ficheiros.

1.7. Resultados Globais Obtidos

Indicador	Resultado confirmado
Aplicações existentes	25
Aplicações com departamento responsável	25
Incidentes existentes	50 000
Incidentes com técnico atribuído	45 455
Incidentes sem técnico atribuído	4 545
Incidentes por aplicação	2 000
Registos selecionados no Exercício 04	75
Aplicações representadas no Exercício 04	25
Meses analisados no Exercício 06	33
Acumulado final de incidentes	50 000
Exercícios implementados e validados	6
Scripts consolidados e conferidos	6

A análise de ranking revelou distribuição uniforme de incidentes pelas aplicações. Para demonstrar a diferença entre RANK() e DENSE_RANK(), foi utilizado um conjunto complementar de valores temporários. A análise temporal confirmou a continuidade dos 33 meses entre janeiro de 2023 e setembro de 2025. Embora a consulta esteja preparada para incluir meses sem incidentes, não foram identificados períodos vazios.

1.8. Delimitações e Considerações Técnicas

O Hands-On tem natureza demonstrativa e utiliza exclusivamente dados fictícios. O objetivo principal é a construção e validação funcional de consultas SQL, não constituindo estudo formal de desempenho ou de otimização de índices. Durante a execução do Exercício 04, foi observada alguma demora na obtenção dos resultados; a consulta terminou corretamente, mas o desempenho não foi diagnosticado. Esta observação poderá ser explorada no HANDS-ON #07 — Índices e Otimização de Queries.

1.9. Resultado Esperado do Hands-On

- Seis scripts T-SQL independentes, comentados e validados.
- Consultas de validação associadas a cada exercício.
- Documentação técnica com cenários, implementação, resultados e boas práticas.
- Evidências de execução no SQL Server Management Studio.
- Organização dos materiais no repositório GitHub do projeto.

2. Apresentação da SIManagerDB e Cenários Funcionais

2.1. Enquadramento da Base de Dados

A SIManagerDB — Gestão de Serviços e Aplicações é uma base de dados relacional demonstrativa, desenvolvida para representar um ambiente organizacional de gestão de sistemas de informação. O modelo contempla estrutura organizacional, aplicações empresariais, equipas técnicas, contratos, licenciamento, incidentes, pedidos de serviço, intervenções e alterações aplicacionais. Foi utilizada no HANDS-ON #05 para explorar o modelo relacional e regras de integridade e é reutilizada no HANDS-ON #06 para consultas e análise em T-SQL. Todos os dados são fictícios.

2.2. Caracterização Geral

Característica	Descrição
SGBD	Microsoft SQL Server
Base de dados	SIManagerDB
Domínio funcional	Gestão de Serviços e Aplicações
Modelo	Relacional
Tabelas funcionais	15
Chaves primárias	15
Chaves estrangeiras	21
Constraints UNIQUE	11
Constraints DEFAULT	2
Constraints CHECK	15
Dados	Fictícios
Ferramenta	SQL Server Management Studio

2.3. Entidades Funcionais da SIManagerDB

Domínio	Tabela	Finalidade
Estrutura organizacional	Departamento	Departamentos da organização
Estrutura organizacional	Equipa	Equipas associadas aos departamentos
Estrutura organizacional	Utilizador	Utilizadores dos sistemas de informação
Estrutura organizacional	Tecnico	Técnicos responsáveis pelo suporte
Gestão de aplicações	Aplicacao	Aplicações empresariais e características
Gestão de aplicações	ModuloAplicacao	Módulos associados às aplicações
Gestão de aplicações	VersaoAplicacao	Versões das aplicações
Gestão contratual	Fornecedor	Fornecedores
Gestão contratual	Contrato	Contratos de aplicações e fornecedores
Gestão contratual	Licenca	Informação de licenciamento
Gestão operacional	PoliticaSLA	Políticas de níveis de serviço
Gestão operacional	Incidente	Incidentes associados às aplicações
Gestão operacional	PedidoServico	Pedidos de serviço
Gestão operacional	Intervencao	Intervenções técnicas sobre incidentes
Gestão operacional	Alteracao	Alterações aplicacionais

O HANDS-ON #06 concentra-se principalmente em Aplicacao, Departamento, Incidente e Tecnico, recorrendo também a Utilizador para identificar os técnicos responsáveis.

2.4. Relacionamentos Relevantes

Origem	Destino	Chave estrangeira	Natureza
Aplicacao	Departamento	DepartamentoResponsavelId	Obrigatório
Incidente	Aplicacao	AplicacaoId	Obrigatório
Incidente	Tecnico	TecnicoId	Opcional
Tecnico	Utilizador	UtilizadorId	Associado

Nos relacionamentos obrigatórios, INNER JOIN recupera os registos com correspondência. No relacionamento opcional entre Incidente e Tecnico, TecnicoId pode ser NULL; LEFT JOIN permite preservar incidentes ainda não atribuídos. A seleção do tipo de junção deve refletir os requisitos funcionais.

2.5. Dados Demonstrativos Utilizados

Entidade ou indicador	Quantidade
Departamentos	12
Aplicações	25
Técnicos	120
Incidentes	50 000
Incidentes atribuídos	45 455
Incidentes não atribuídos	4 545
Incidentes por aplicação	2 000

A distribuição dos incidentes pelas aplicações é uniforme: 2 000 incidentes por aplicação. O intervalo temporal analisado vai de janeiro de 2023 a setembro de 2025, num total de 33 meses consecutivos.

2.6. Cenário Funcional 01 — Aplicações e Departamentos Responsáveis

Objetivo: Identificar as aplicações empresariais e os departamentos responsáveis pela sua gestão.

Tabelas envolvidas: dbo.Aplicacao; dbo.Departamento

Técnicas principais: INNER JOIN

Resultado validado: 25 aplicações existentes, 25 com correspondência e 0 sem departamento responsável. Demonstra junções internas em relacionamentos obrigatórios.

2.7. Cenário Funcional 02 — Incidentes com e sem Técnico Atribuído

Objetivo: Identificar os incidentes e distinguir os que possuem técnico responsável daqueles sem atribuição.

Tabelas envolvidas: dbo.Incidente; dbo.Tecnico; dbo.Utilizador

Técnicas principais: LEFT JOIN; CASE

Resultado validado: 50 000 incidentes: 45 455 atribuídos e 4 545 não atribuídos. A junção externa preservou todos os registos.

2.8. Cenário Funcional 03 — Indicadores de Incidentes por Aplicação

Objetivo: Calcular o número de incidentes associados a cada aplicação.

Tabelas envolvidas: dbo.Aplicacao; dbo.Incidente

Técnicas principais: CTE; LEFT JOIN; GROUP BY; COUNT()

Resultado validado: 25 aplicações, 50 000 incidentes, 2 000 por aplicação e 0 aplicações sem incidentes.

2.9. Cenário Funcional 04 — Últimos Três Incidentes por Aplicação

Objetivo: Identificar os três incidentes mais recentes de cada aplicação, mantendo o detalhe.

Tabelas envolvidas: dbo.Incidente; dbo.Aplicacao

Técnicas principais: CTE; ROW_NUMBER(); PARTITION BY; COUNT() OVER()

Resultado validado: 75 linhas, 25 aplicações, três incidentes por aplicação e total de 2 000 por aplicação. Ordenação determinística por DataAbertura e IncidenteId.

2.10. Cenário Funcional 05 — Ranking de Aplicações

Objetivo: Classificar as aplicações pelo número de incidentes e comparar o comportamento perante empates.

Tabelas envolvidas: dbo.Aplicacao; dbo.Incidente

Técnicas principais: CTE; GROUP BY; RANK(); DENSE_RANK()

Resultado validado: Todas as 25 aplicações apresentam 2 000 incidentes e posição 1. Uma demonstração com VALUES mostrou que RANK() salta posições após empates, ao contrário de DENSE_RANK().

2.11. Cenário Funcional 06 — Evolução Mensal dos Incidentes

Objetivo: Analisar totais mensais, acumulados e valores dos meses anterior e seguinte.

Tabelas envolvidas: dbo.Incidente

Técnicas principais: CTEs; GROUP BY; SUM() OVER(); LAG(); LEAD(); LEFT JOIN; CTE recursiva

Resultado validado: 33 meses consecutivos, janeiro de 2023 a setembro de 2025; acumulado final de 50 000; nenhum mês sem incidentes. A consulta está preparada para meses vazios, mas essa situação não ocorreu.

2.12. Síntese dos Cenários Funcionais

Exercício	Abordagem	Resultado principal
01	Junção interna	25 aplicações com departamento
02	Junção externa	50 000 incidentes preservados
03	Agregação com CTE	2 000 incidentes por aplicação
04	Numeração por grupo	75 incidentes selecionados
05	Classificação e empates	Ranking e demonstração complementar
06	Análise temporal	33 meses e acumulado de 50 000

3. Implementação Técnica dos Exercícios SQL

3.1. Abordagem de Implementação

A implementação foi organizada em seis exercícios independentes e progressivos, executados em T-SQL sobre a SIManagerDB. Cada ficheiro inclui a consulta principal e verificações complementares. Todas as operações são de leitura. Os seis ficheiros consolidados foram novamente executados e conferidos no SSMS.

3.2. Exercício 01 — Aplicações e Departamentos Responsáveis

Objetivo técnico: relacionar aplicações empresariais e departamentos responsáveis através de INNER JOIN. Tabelas: dbo.Aplicacao e dbo.Departamento.

```
USE SIManagerDB;
GO
/* Exercicio 01 – INNER JOIN */
SELECT
    a.AplicacaoId,
    a.Nome AS Aplicacao,
    a.Criticidade,
    a.Estado,
    d.Nome AS DepartamentoResponsavel
FROM dbo.Aplicacao AS a
INNER JOIN dbo.Departamento AS d
    ON a.DepartamentoResponsavelId = d.DepartamentoId
ORDER BY a.Nome, a.AplicacaoId;
```

INNER JOIN relaciona as tabelas pelas chaves de correspondência. Os aliases identificam as tabelas de origem. Resultado: 25 aplicações, todas com departamento, e 0 sem correspondência. Ficheiro: 01_InnerJoin_Aplicacoes_Departamentos.sql.

3.3. Exercício 02 — Incidentes com e sem Técnico Atribuído

Objetivo técnico: preservar todos os incidentes, mesmo quando não existe técnico atribuído. Tabelas: dbo.Incidente, dbo.Tecnico e dbo.Utilizador.

```
USE SIManagerDB;
GO
/* Exercicio 02 – LEFT JOIN */
SELECT
    i.IncidenteId,
    i.Titulo,
    i.Estado,
    i.DataAbertura,
    i.TecnicoId,
    u.Nome AS TecnicoResponsavel,
    CASE
        WHEN i.TecnicoId IS NULL THEN 'Nao Atribuido'
        ELSE 'Atribuido'
    END AS SituacaoAtribuicao
FROM dbo.Incidente AS i
LEFT JOIN dbo.Tecnico AS t
    ON i.TecnicoId = t.TecnicoId
LEFT JOIN dbo.Utilizador AS u
    ON t.UtilizadorId = u.UtilizadorId
ORDER BY i.IncidenteId;
```

LEFT JOIN preserva os incidentes sem técnico. CASE distingue as situações de atribuição. Resultado: 50 000 incidentes, 45 455 atribuídos e 4 545 não atribuídos. Ficheiro: 02_LeftJoin_Incidentes_Tecnicos.sql.

3.4. Exercício 03 — Indicadores de Incidentes por Aplicação

Objetivo técnico: calcular incidentes por aplicação com CTE e agregação. Tabelas: dbo.Aplicacao e dbo.Incidente.

```
USE SIManagerDB;
GO
/* Exercicio 03 – CTE + GROUP BY */
;WITH CTE_IncidentesPorAplicacao AS
(
    SELECT
        a.AplicacaoId,
        a.Nome AS Aplicacao,
        a.Criticidade,
        COUNT(i.IncidenteId) AS TotalIncidentes
    FROM dbo.Aplicacao AS a
    LEFT JOIN dbo.Incidente AS i
        ON a.AplicacaoId = i.AplicacaoId
    GROUP BY a.AplicacaoId, a.Nome, a.Criticidade
)
SELECT
    AplicacaoId, Aplicacao, Criticidade, TotalIncidentes
FROM CTE_IncidentesPorAplicacao
ORDER BY TotalIncidentes DESC, AplicacaoId ASC;
```

A CTE separa a agregação da apresentação. LEFT JOIN inclui aplicações sem incidentes; COUNT(i.IncidenteId) devolve zero para correspondências inexistentes. Resultado: 25 aplicações, 2 000 incidentes por aplicação, total de 50 000. Ficheiro: 03_CTE_Incidentes_Por_Aplicacao.sql.

3.5. Exercício 04 — Últimos Três Incidentes por Aplicação

Objetivo técnico: selecionar os três incidentes mais recentes por aplicação e preservar o detalhe e o total do grupo. Tabelas: dbo.Incidente e dbo.Aplicacao.

```
USE SIManagerDB;
GO
/* Exercicio 04 – ROW_NUMBER() + COUNT() OVER() */
;WITH CTE_IncidentesOrdenados AS
(
    SELECT
        a.AplicacaoId,
        a.Nome AS Aplicacao,
        i.IncidenteId,
        i.Titulo,
        i.Estado,
        i.DataAbertura,
        ROW_NUMBER() OVER
        (
            PARTITION BY i.AplicacaoId
            ORDER BY i.DataAbertura DESC, i.IncidenteId DESC
        ) AS Posicao,
        COUNT(*) OVER
        (
            PARTITION BY i.AplicacaoId
        ) AS TotalIncidentesAplicacao
    FROM dbo.Incidente AS i
    INNER JOIN dbo.Aplicacao AS a
        ON i.AplicacaoId = a.AplicacaoId
)
SELECT
    AplicacaoId, Aplicacao, IncidenteId, Titulo,
    Estado, DataAbertura, Posicao, TotalIncidentesAplicacao
FROM CTE_IncidentesOrdenados
WHERE Posicao <= 3
ORDER BY AplicacaoId, Posicao;
```

ROW_NUMBER() numera cada aplicação; IncidenteId desempata datas iguais e assegura ordenação determinística. COUNT() OVER() conserva o total de cada grupo. O filtro é aplicado na consulta

exterior. Resultado: 75 linhas, 25 aplicações, posições 1 a 3, total de 2 000 incidentes por aplicação. Foi observada demora inicial, sem diagnóstico formal de desempenho. Ficheiro: 04_RowNumber_Ultimos_Incidentes.sql.

3.6. Exercício 05 — Ranking de Aplicações por Volume de Incidentes

Objetivo técnico: classificar aplicações e comparar RANK() e DENSE_RANK(). Tabelas: dbo.Aplicacao e dbo.Incidente.

```
USE SIManagerDB;
GO
/* Exercicio 05 – RANK() + DENSE_RANK() */
;WITH CTE_IncidentesPorAplicacao AS
(
    SELECT
        a.AplicacaoId,
        a.Nome AS Aplicacao,
        a.Criticidade,
        COUNT(i.IncidenteId) AS TotalIncidentes
    FROM dbo.Aplicacao AS a
    LEFT JOIN dbo.Incidente AS i
        ON a.AplicacaoId = i.AplicacaoId
    GROUP BY a.AplicacaoId, a.Nome, a.Criticidade
),
CTE_RankingAplicacoes AS
(
    SELECT
        AplicacaoId, Aplicacao, Criticidade, TotalIncidentes,
        RANK() OVER
        (
            ORDER BY TotalIncidentes DESC
        ) AS PosicaoRank,
        DENSE_RANK() OVER
        (
            ORDER BY TotalIncidentes DESC
        ) AS PosicaoDenseRank
    FROM CTE_IncidentesPorAplicacao
)
SELECT
    AplicacaoId, Aplicacao, Criticidade, TotalIncidentes,
    PosicaoRank, PosicaoDenseRank
FROM CTE_RankingAplicacoes
ORDER BY PosicaoRank, AplicacaoId;
```

Todas as 25 aplicações apresentam 2 000 incidentes, recebendo posição 1 em ambas as funções. Para mostrar a diferença nos empates foi utilizada a demonstração complementar abaixo, sem modificar a SIManagerDB.

```
;WITH CTE_DadosExemplo AS
(
    SELECT Aplicacao, TotalIncidentes
    FROM
    (
        VALUES
            ('Aplicacao A', 2500),
            ('Aplicacao B', 2000),
            ('Aplicacao C', 2000),
            ('Aplicacao D', 1500),
            ('Aplicacao E', 1000)
    ) AS Dados(Aplicacao, TotalIncidentes)
)
SELECT
    Aplicacao, TotalIncidentes,
    RANK() OVER
    (
        ORDER BY TotalIncidentes DESC
    ) AS PosicaoRank,
    DENSE_RANK() OVER
    (
        ORDER BY TotalIncidentes DESC
    ) AS PosicaoDenseRank
FROM CTE_DadosExemplo
ORDER BY TotalIncidentes DESC, Aplicacao;
```

Aplicação	Incidentes	RANK	DENSE_RANK
Aplicacao A	2 500	1	1
Aplicacao B	2 000	2	2
Aplicacao C	2 000	2	2
Aplicacao D	1 500	4	3
Aplicacao E	1 000	5	4

RANK() salta a posição 3 após o empate na posição 2; DENSE_RANK() mantém posições consecutivas. Não se adiciona AplicacaoId ao ORDER BY das funções de ranking para não destruir os empates. Ficheiro: 05_Ranking_Aplicacoes.sql.

3.7. Exercício 06 — Evolução Mensal dos Incidentes

Objetivo técnico: construir uma série mensal contínua, com totais, acumulados, valores do mês anterior e seguinte e diferenças mensais. Tabela: dbo.Incidente.

```
USE SIManagerDB;
GO
/* Exercicio 06 – Evolucao Mensal dos Incidentes */
;WITH CTE_Limites AS
(
    SELECT
        DATEFROMPARTS(
            YEAR(MIN(DataAbertura)),
            MONTH(MIN(DataAbertura)), 1
        ) AS PrimeiroMes,
        DATEFROMPARTS(
            YEAR(MAX(DataAbertura)),
            MONTH(MAX(DataAbertura)), 1
        ) AS UltimoMes
    FROM dbo.Incidente
),
CTE_CalendarioMensal AS
(
    SELECT PrimeiroMes AS MesReferencia, UltimoMes
    FROM CTE_Limites
    WHERE PrimeiroMes IS NOT NULL
    UNION ALL
    SELECT DATEADD(MONTH, 1, MesReferencia), UltimoMes
    FROM CTE_CalendarioMensal
    WHERE MesReferencia < UltimoMes
),
CTE_IncidentesMensais AS
(
    SELECT
        DATEFROMPARTS(
            YEAR(DataAbertura), MONTH(DataAbertura), 1
        ) AS MesReferencia,
        COUNT(*) AS TotalIncidentes
    FROM dbo.Incidente
    GROUP BY DATEFROMPARTS(
        YEAR(DataAbertura), MONTH(DataAbertura), 1
    )
```

```
),
CTE_SerieMensal AS
(
    SELECT
        c.MesReferencia,
        COALESCE(m.TotalIncidentes, 0) AS TotalIncidentes
    FROM CTE_CalendarioMensal AS c
    LEFT JOIN CTE_IncidentesMensais AS m
        ON c.MesReferencia = m.MesReferencia
),
CTE_AnaliseMensal AS
(
    SELECT
        MesReferencia, TotalIncidentes,
        SUM(TotalIncidentes) OVER
        (
            ORDER BY MesReferencia
            ROWS BETWEEN UNBOUNDED PRECEDING
                     AND CURRENT ROW
        ) AS TotalAcumulado,
        LAG(TotalIncidentes) OVER
        (
            ORDER BY MesReferencia
        ) AS IncidentesMesAnterior,
        LEAD(TotalIncidentes) OVER
        (
            ORDER BY MesReferencia
        ) AS IncidentesMesSeguinte
    FROM CTE_SerieMensal
)
SELECT
    YEAR(MesReferencia) AS Ano,
    MONTH(MesReferencia) AS Mes,
    MesReferencia,
    TotalIncidentes,
    TotalAcumulado,
    IncidentesMesAnterior,
```

```
    CASE
        WHEN IncidentesMesAnterior IS NULL THEN NULL
        ELSE TotalIncidentes - IncidentesMesAnterior
    END AS DiferencaMesAnterior,
    IncidentesMesSeguinte
FROM CTE_AnaliseMensal
ORDER BY MesReferencia
OPTION (MAXRECURSION 0);
```

CTE_Limites identifica o primeiro e último mês; CTE_CalendarioMensal gera os meses consecutivos; CTE_IncidentesMensais agrega os incidentes; CTE_SerieMensal relaciona calendário e totais, preenchendo faltas com COALESCE; CTE_AnaliseMensal aplica SUM() OVER(), LAG() e LEAD(). MAXRECURSION 0 remove o limite padrão de recursão, mantendo a condição de paragem no último mês.

Resultado: 33 meses, janeiro de 2023 a setembro de 2025; 50 000 incidentes e acumulado final de 50 000; zero meses sem incidentes. A lógica para meses vazios está implementada, mas não foi exercitada. Ficheiro: 06_Evolucao_Mensal_Incidentes.sql.

3.8. Organização dos Scripts SQL

```
hands-on-06/
└── scripts/
    ├── 01_InnerJoin_Aplicacoes_Departamentos.sql
    ├── 02_LeftJoin_Incidentes_Tecnicos.sql
    ├── 03_CTE_Incidentes_Por_Aplicacao.sql
    ├── 04_RowNumber_Ultimos_Incidentes.sql
    ├── 05_Ranking_Aplicacoes.sql
    └── 06_Evolucao_Mensal_Incidentes.sql
```

Cada ficheiro inclui identificação institucional e do exercício, consulta principal, consulta ou demonstração de validação e comentários técnicos em português de Portugal. Os seis ficheiros foram reexecutados e conferidos no SSMS.

3.9. Síntese Técnica da Implementação

Exercício	Técnica principal	Resultado
01	INNER JOIN	25 aplicações com departamento
02	LEFT JOIN	50 000 incidentes preservados
03	CTE + GROUP BY	2 000 incidentes por aplicação
04	ROW_NUMBER() + COUNT() OVER()	75 incidentes selecionados
05	RANK() + DENSE_RANK()	Classificação e empates
06	SUM() OVER() + LAG() + LEAD()	33 meses; acumulado 50 000

4. Validação Técnica e Evidências de Execução

4.1. Objetivo da Validação

Confirmar a correção funcional das consultas T-SQL desenvolvidas sobre a SIManagerDB: correspondências entre tabelas, preservação de registos opcionais, agregações, funções de janela, empates, continuidade temporal, reconciliação de indicadores e execução dos scripts consolidados. As verificações basearam-se em consultas complementares e na análise dos resultados apresentados no SSMS.

4.2. Metodologia de Validação

Fase 1 — Validação individual: cada exercício foi executado no SSMS e verificado com consultas complementares, com recolha de capturas dos resultados. Fase 2 — Conferência dos scripts consolidados: os seis ficheiros independentes foram novamente executados e os indicadores confrontados com os resultados anteriores. Resultado global: seis exercícios implementados, validados e conferidos.

4.3. Exercício 01 — Validação do INNER JOIN

Indicador	Resultado
TotalAplicacoes	25
TotalComCorrespondencia	25
AplicacoesSemDepartamento	0

As 25 aplicações possuem correspondência com departamentos. A consulta não excluiu aplicações por ausência de correspondência. Estado: VALIDADO. Evidência: captura SSMS com indicadores 25 | 25 | 0.

4.4. Exercício 02 — Validação do LEFT JOIN

Indicador	Resultado
TotalIncidentes	50 000
IncidentesAtribuidos	45 455
IncidentesNaoAtribuidos	4 545
TotalComLeftJoin	50 000

45 455 + 4 545 = 50 000 incidentes. A junção preservou todos os incidentes sem perda ou multiplicação de linhas. Estado: VALIDADO. Evidência: SSMS com registos atribuídos e não atribuídos e os quatro indicadores.

4.5. Exercício 03 — Validação da CTE e GROUP BY

Indicador	Resultado
TotalAplicacoes	25
AplicacoesNaCTE	25
TotalIncidentesBD	50 000
TotalIncidentesCTE	50 000
AplicacoesSemIncidentes	0

25 × 2 000 = 50 000. Os valores agregados reconciliam com os dados de origem. COUNT(i.IncidenteId) permite representar zero quando não existem incidentes associados. Estado: VALIDADO. Evidência: SSMS com totais por aplicação e linha de validação.

4.6. Exercício 04 — Validação do ROW_NUMBER() e COUNT() OVER()

Indicador	Resultado
TotalLinhas	75
AplicacoesRepresentadas	25
PosicaoMinima	1
PosicaoMaxima	3
MinIncidentesAplicacao	2 000
MaxIncidentesAplicacao	2 000

25 × 3 = 75 linhas. A numeração reinicia por aplicação, e o total de incidentes é preservado em cada registo selecionado. Estado: VALIDADO. Foi observada alguma demora inicial, sem análise formal de desempenho. Evidência: SSMS com os incidentes e indicadores 75 | 25 | 1 | 3 | 2000 | 2000.

4.7. Exercício 05 — Validação do RANK() e DENSE_RANK()

As 25 aplicações possuem 2 000 incidentes e posição 1 em ambas as funções. Uma consulta complementar com VALUES demonstrou empates e diferenças na sequência de classificação.

Aplicação	Incidentes	RANK	DENSE_RANK
Aplicacao A	2 500	1	1
Aplicacao B	2 000	2	2
Aplicacao C	2 000	2	2
Aplicacao D	1 500	4	3
Aplicacao E	1 000	5	4

Após empate na posição 2, RANK() atribui posição 4 e DENSE_RANK() atribui posição 3. A demonstração não alterou a SIManagerDB. Estado: VALIDADO. Evidências: capturas da classificação principal e da demonstração complementar.

4.8. Exercício 06 — Validação da Evolução Mensal dos Incidentes

Indicador	Resultado
PrimeiroMes	2023-01-01
UltimoMes	2025-09-01
TotalMeses	33
TotalIncidentesSerie	50 000
TotalIncidentesBD	50 000
AcumuladoFinal	50 000
MesesSemIncidentes	0

A série possui 33 meses consecutivos. Soma mensal, total na tabela de origem e acumulado final coincidem em 50 000. LAG() devolve NULL no primeiro mês e LEAD() devolve NULL no último. A lógica de meses sem incidentes foi implementada, mas não exercitada com dados de total zero. Estado: VALIDADO.

Exemplo de comparação mensal apresentado na documentação aprovada:

Mês	Incidentes	Acumulado	Diferença mensal
Janeiro de 2023	1 550	1 550	—
Fevereiro de 2023	1 400	2 950	-150
Março de 2023	1 550	4 500	+150
Abril de 2023	1 500	6 000	-50

4.9. Conferência dos Scripts SQL Consolidados

Script	Técnica	Estado
01	INNER JOIN	Validado
02	LEFT JOIN	Validado
03	CTE + GROUP BY	Validado
04	ROW_NUMBER() + COUNT() OVER()	Validado
05	RANK() + DENSE_RANK()	Validado
06	SUM() OVER() + LAG() + LEAD()	Validado

A reexecução dos ficheiros consolidados confirmou a coerência dos resultados com os valores obtidos durante o desenvolvimento.

4.10. Critérios Globais de Aceitação

Critério	Resultado
Execução das seis consultas principais	Cumprido
Validação dos relacionamentos	Cumprido
Preservação dos incidentes sem técnico	Cumprido
Reconciliação dos totais agregados	Cumprido
Validação das posições por aplicação	Cumprido
Demonstração de empates no ranking	Cumprido
Reconciliação da série temporal	Cumprido
Conferência dos seis scripts	Cumprido
Demonstração de mês sem incidentes	Não exercitada
Análise formal de desempenho	Fora do âmbito

4.11. Limitações e Considerações Técnicas

A distribuição uniforme de 2 000 incidentes por aplicação não produz rankings distintos na consulta principal. Não existem meses vazios na série temporal de referência. Foi observada demora no Exercício 04, sem métricas suficientes para atribuir uma causa. Estas limitações devem permanecer explícitas na interpretação dos resultados.

4.12. Conclusão da Validação Técnica

As consultas e as verificações complementares confirmaram a coerência dos resultados com os dados de origem: 25 aplicações com departamento, 50 000 incidentes preservados, 2 000 por aplicação, 75 incidentes selecionados, ranking com empates demonstrado e 33 meses com acumulado final de 50 000. Os seis scripts foram conferidos. Conclui-se que os exercícios cumprem os objetivos funcionais e técnicos dentro do âmbito documentado.

5. Boas Práticas, Considerações Técnicas e Conclusões

5.1. Enquadramento

O desenvolvimento do HANDS-ON #06 privilegiou correção funcional, clareza técnica e validação dos resultados. As boas práticas resultam das técnicas aplicadas e das observações efetuadas durante a implementação.

5.2. Seleção Adequada dos Tipos de JOIN

INNER JOIN: utilizar quando o resultado exige correspondência entre entidades relacionadas; no Exercício 01, todas as 25 aplicações possuem departamento. LEFT JOIN: utilizar quando se pretende preservar os registos da entidade principal sem correspondência opcional; no Exercício 02, 4 545 incidentes não têm técnico atribuído. Atenção aos filtros na cláusula WHERE sobre tabelas relacionadas: podem excluir linhas com NULL e alterar o efeito da junção externa. A localização dos filtros em ON ou WHERE deve respeitar o objetivo funcional.

5.3. Organização de Consultas com CTEs

As CTEs separaram agregação, numeração, rankings, calendários e indicadores temporais. No Exercício 06, cinco CTEs com responsabilidades distintas facilitaram a leitura e validação da lógica. Uma CTE não é armazenamento permanente nem garante materialização ou melhoria automática de desempenho; a avaliação depende do plano de execução, índices, volumes e operações.

5.4. Agregações e Preservação dos Resultados

No Exercício 03, COUNT(i.IncidenteId) contabiliza apenas identificadores não nulos. Em LEFT JOIN sem correspondência, COUNT(*) contabilizaria a linha produzida pela junção, enquanto COUNT(i.IncidenteId) devolve zero. A expressão de agregação deve refletir o significado funcional. A soma dos totais por aplicação reconciliou com 50 000 incidentes.

```
COUNT(i.IncidenteId)
```

5.5. Funções de Janela e Preservação do Detalhe

Funções de janela calculam indicadores sobre grupos sem reduzir necessariamente o detalhe dos registos. COUNT(*) OVER (PARTITION BY i.AplicacaoId) devolve o total por aplicação preservando os incidentes individuais. PARTITION BY deve refletir a granularidade funcional pretendida.

```
COUNT(*) OVER
(
    PARTITION BY i.AplicacaoId
)
```

5.6. Ordenação Determinística com ROW_NUMBER()

A ordenação por DataAbertura DESC e IncidenteId DESC resolve empates de datas e permite selecionar os três incidentes mais recentes de modo determinístico, considerando IncidenteId como chave primária. A ordenação da função de janela é distinta da ordenação final apresentada ao utilizador.

```
ORDER BY
    i.DataAbertura DESC,
    i.IncidenteId DESC
```

5.7. Tratamento de Empates com RANK() e DENSE_RANK()

RANK() pode produzir lacunas após empates; DENSE_RANK() mantém posições distintas consecutivas. Como todas as aplicações possuem 2 000 incidentes, foi utilizada demonstração complementar com cinco valores. Adicionar um identificador único ao ORDER BY das funções de ranking poderia eliminar os empates que se pretendem representar.

5.8. Análise Temporal com SUM(), LAG() e LEAD()

SUM() OVER() calculou o acumulado mensal com enquadramento ROWS explícito; LAG() recuperou o valor do mês anterior; LEAD() recuperou o valor do mês seguinte. Uma série temporal deve ter granularidade consistente e ordenação cronológica antes de comparar períodos.

```
ROWS BETWEEN UNBOUNDED PRECEDING
         AND CURRENT ROW
```

5.9. Continuidade das Séries Temporais

Uma agregação apenas sobre registos existentes pode omitir meses vazios. No Exercício 06, uma CTE recursiva gerou o calendário, LEFT JOIN associou os totais e COALESCE apresentou zero para faltas. A série de 33 meses não continha meses sem incidentes; essa situação foi prevista na implementação, mas não foi exercitada.

5.10. Validação e Reconciliação de Indicadores

As consultas principais foram acompanhadas de verificações complementares: correspondência de aplicações e departamentos; preservação dos incidentes; reconciliação de totais; posições por aplicação; empates de ranking; e reconciliação temporal. A execução sem erros não basta para considerar uma consulta validada: devem existir critérios mensuráveis e verificação da coerência funcional.

5.11. Legibilidade e Manutenção do Código T-SQL

• Utilizar aliases para identificar tabelas.

• Atribuir nomes descritivos a CTEs e colunas calculadas.

• Separar operações em etapas lógicas.

• Manter indentação consistente e comentários técnicos.

• Definir ordenação explícita quando necessária.

• Organizar exercícios em ficheiros independentes, com consultas principais e validações.

5.12. Considerações sobre Desempenho

O HANDS-ON #06 priorizou implementação funcional e validação, sem estudo formal de otimização. Foi observada demora inicial no Exercício 04, mas não foram recolhidas métricas para identificar a causa. As funções de janela podem exigir ordenação e processamento de conjuntos maiores do que o resultado final; o impacto deve ser confirmado por planos de execução, estatísticas de tempo e leituras, avaliação de índices e comparação de alternativas. Estas atividades enquadram-se no HANDS-ON #07.

5.13. Limitações do Ambiente Demonstrativo

A SIManagerDB contém dados fictícios com distribuição uniforme de incidentes, ausência de meses vazios no intervalo temporal e sem avaliação formal de desempenho. As limitações não invalidam os resultados confirmados, mas devem ser consideradas na sua interpretação e reutilização.

5.14. Resultados Globais do Hands-On

Indicador	Resultado
Exercícios implementados	6
Exercícios validados	6
Scripts consolidados e conferidos	6
Aplicações analisadas	25
Incidentes existentes	50 000
Incidentes com técnico atribuído	45 455
Incidentes sem técnico atribuído	4 545
Incidentes por aplicação	2 000
Registos selecionados com ROW_NUMBER()	75
Meses analisados	33
Acumulado final de incidentes	50 000

5.15. Conclusões

O HANDS-ON #06 demonstrou Joins, CTEs, agregações e funções de janela em cenários funcionais de gestão de serviços e aplicações. A SIManagerDB permitiu ligar o modelo relacional do HANDS-ON #05 à exploração da informação. Os exercícios relacionaram entidades, preservaram registos opcionais, produziram indicadores, selecionaram registos por posição, construíram rankings e analisaram evolução temporal. Os seis scripts foram validados e conferidos. A qualidade de uma consulta SQL depende da adequação funcional, clareza da implementação e validação dos resultados, não apenas da execução sem erros.

6. Anexos e Organização do Repositório GitHub

6.1. Objetivo

A organização dos materiais técnicos facilita a reprodução, consulta e reutilização dos exemplos T-SQL, a verificação dos resultados, a manutenção das evidências e a continuidade da Série 2. A SIManagerDB constitui a base demonstrativa comum.

6.2. Organização Geral do Repositório

```
SIManagerDB/
├── README.md
├── database/
│   ├── SIManagerDB_Instalacao.sql
│   └── SIManagerDB_Validacao.sql
├── hands-on-05/
│   ├── README.md
│   ├── scripts/
│   │   ├── 01_Consultar_Relacionamentos.sql
│   │   ├── 02_Consultar_Constraints.sql
│   │   └── 03_Testes_Integridade.sql
│   └── docs/
│       ├── HandsOn05_Documentacao.pdf
│       ├── SIManagerDB_DicionarioDados_Final.csv
│       └── SIManagerDB_DiagramaRelacional.png
├── hands-on-06/
│   ├── README.md
│   ├── scripts/
│   │   ├── 01_InnerJoin_Aplicacoes_Departamentos.sql
│   │   ├── 02_LeftJoin_Incidentes_Tecnicos.sql
│   │   ├── 03_CTE_Incidentes_Por_Aplicacao.sql
│   │   ├── 04_RowNumber_Ultimos_Incidentes.sql
│   │   ├── 05_Ranking_Aplicacoes.sql
│   │   └── 06_Evolucao_Mensal_Incidentes.sql
│   └── docs/
│       └── HandsOn06_Documentacao.pdf
├── hands-on-07/
│   └── ...
├── hands-on-08/
│   └── ...
└── hands-on-09/
    └── ...
```

Nota técnica: A estrutura representa a organização prevista. A referência a um ficheiro não confirma a sua publicação no GitHub. O PDF será integrado em docs/ após aprovação e geração.

6.3. Base de Dados Demonstrativa

Ficheiro	Finalidade
SIManagerDB_Instalacao.sql	Criar base de dados, tabelas, relacionamentos e dados fictícios
SIManagerDB_Validacao.sql	Verificar estrutura e indicadores principais

A SIManagerDB instalada e povoada é pré-requisito. A versão reforçada do script de instalação preparada no HANDS-ON #05 deverá ser distinguida de versões cuja execução já foi efetivamente confirmada; a disponibilização pública de nova versão deve ser precedida de testes.

6.4. Scripts do HANDS-ON #06

Ficheiro	Conteúdo
01_InnerJoin_Aplicacoes_Departamentos.sql	Aplicações e departamentos com INNER JOIN
02_LeftJoin_Incidentes_Tecnicos.sql	Incidentes atribuídos e não atribuídos com LEFT JOIN
03_CTE_Incidentes_Por_Aplicacao.sql	Agregação com CTE e GROUP BY
04_RowNumber_Ultimos_Incidentes.sql	Três incidentes mais recentes com ROW_NUMBER()
05_Ranking_Aplicacoes.sql	RANK(), DENSE_RANK() e demonstração de empates
06_Evolucao_Mensal_Incidentes.sql	SUM() OVER(), LAG(), LEAD() e calendário mensal

6.5. Estrutura Interna dos Ficheiros SQL

Os scripts contemplam cabeçalho de identificação institucional, contextualização funcional, consulta principal, validação técnica e comentários. A estrutura facilita a execução independente e a comparação com os resultados documentados.

6.6. Documentação Técnica

A pasta hands-on-06/docs/ destina-se ao documento HandsOn06_Documentacao.pdf, que reúne as seis secções aprovadas: Introdução e Objetivos; Apresentação da SIManagerDB e Cenários Funcionais; Implementação Técnica dos Exercícios SQL; Validação Técnica e Evidências de Execução; Boas Práticas, Considerações Técnicas e Conclusões; Anexos e Organização do Repositório GitHub. A designação Ponto De Conhecimento Tecnológico deve constar dos cabeçalhos e rodapés.

6.7. Evidências de Execução

Exercício	Evidência principal
01	Aplicações e departamentos; 25 correspondências
02	Incidentes atribuídos/não atribuídos; total de 50 000
03	Agregação por aplicação; reconciliação de 50 000
04	Últimos três incidentes; 75 linhas
05	Ranking principal e demonstração dos empates
06	Evolução mensal e acumulado final de 50 000

As capturas foram analisadas durante a execução e conferência dos scripts. A sua inclusão no PDF deve privilegiar evidências relevantes e evitar repetições. A estrutura opcional de armazenamento separado é:

```
hands-on-06/
└── docs/
    ├── HandsOn06_Documentacao.pdf
    └── evidencias/
        ├── Exercicio01_Validacao.png
        ├── Exercicio02_Validacao.png
        ├── Exercicio03_Validacao.png
        ├── Exercicio04_Validacao.png
        ├── Exercicio05_Validacao.png
        └── Exercicio06_Validacao.png
```

Nota técnica: Os ficheiros de imagem não estão incorporados neste documento porque as capturas originais não foram disponibilizadas como ficheiros nesta etapa. A estrutura de evidências é uma proposta, não uma confirmação de publicação.

6.8. README do HANDS-ON #06

O ficheiro hands-on-06/README.md deverá funcionar como ponto de entrada, incluindo identificação institucional, objetivos, pré-requisitos, resumo dos exercícios, estrutura de diretórios, instruções SSMS, resultados validados, documentação e ligação ao HANDS-ON #05 e à SIManagerDB.

6.9. Pré-requisitos de Execução

Requisito	Descrição
Microsoft SQL Server	Versão 2019 ou superior
SQL Server Management Studio	Ferramenta de execução
SIManagerDB	Instalada e povoada
Permissões	Leitura nas tabelas utilizadas
Scripts SQL	Seis ficheiros do HANDS-ON #06

6.10. Instruções para Reproduzir os Exercícios

• Preparar o Microsoft SQL Server e o SSMS.

• Confirmar a instalação e povoamento da SIManagerDB.

• Abrir no SSMS o ficheiro SQL do exercício pretendido.

• Confirmar que a execução ocorre no contexto da SIManagerDB.

• Executar a consulta principal e observar os resultados.

• Executar a validação técnica incluída no ficheiro.

• Confrontar os indicadores com os valores documentados.

Os exercícios são independentes, desde que os respetivos pré-requisitos estejam assegurados.

6.11. Resultados de Referência

Indicador	Valor
Aplicações	25
Incidentes	50 000
Incidentes atribuídos	45 455
Incidentes não atribuídos	4 545
Incidentes por aplicação	2 000
Registos do Exercício 04	75
Meses analisados	33
Acumulado final	50 000
Exercícios validados	6
Scripts consolidados e conferidos	6

Os valores são referências para o mesmo conjunto de dados fictícios, não resultados universais para qualquer instalação ou povoamento.

6.12. Continuidade da Série 2 — SQL Server

Hands-On	Tema
#05	Modelação de Dados e Constraints
#06	Joins, CTEs e Window Functions
#07	Índices e Otimização de Queries
#08	Transações e Tratamento de Erros
#09	Auditoria de Dados com Triggers e Tabelas Temporais

6.13. Considerações sobre a Publicação no GitHub

• Conferir nomes e caminhos dos ficheiros.

• Garantir a presença dos seis scripts consolidados.

• Verificar coerência entre documentação e resultados validados.

• Disponibilizar a documentação técnica final.

• Confirmar os caminhos relativos do README.

• Identificar os dados como fictícios.

• Excluir credenciais, dados pessoais reais e informação confidencial.

• Distinguir materiais concluídos e conteúdos em preparação.

6.14. Síntese dos Anexos

Material	Situação
Seis scripts SQL	Consolidados e validados
Consultas complementares	Incluídas nos scripts
Evidências SSMS	Recolhidas e analisadas
Documentação técnica	Seis secções aprovadas
PDF técnico	Gerado nesta etapa
README do HANDS-ON #06	Pendente de preparação ou conferência
Publicação no GitHub	Pendente de confirmação

6.15. Conclusão

A organização dos anexos e do repositório facilita a reprodução e a manutenção dos conteúdos. Os seis scripts SQL foram implementados, validados e conferidos no SSMS. A documentação reúne enquadramento funcional, consultas, resultados e boas práticas. A estrutura proposta permite dar continuidade à Série 2 — SQL Server e reutilizar a SIManagerDB nos exercícios seguintes.